GitHub Spec Kit: Team Training Completion Log

Spec-Driven Development (SDD) Self-Training Evidence
Course: CS300 - CSC13002 - Introduction to Software Engineering
Project Assignment: PA2-2026 (Part C Deliverable)
Group: Group 07 - NutriViet Team

1. Overview of Training Activity

In compliance with Section C of the Project Assignment 2 (PA2-2026) specification, all 5 members of the NutriViet engineering team completed the official GitHub Spec Kit video tutorial curriculum on Spec-Driven Development (SDD) between **19/10/2026** and **23/10/2026**.

The training covered:

1. The transition from informal "vibe coding" to structured, reproducible **Spec-Driven Development (SDD)**.
2. The role of the **Constitution (`constitution.md`)** as an authoritative, long-term memory governance file for AI coding agents (GitHub Copilot, Cursor, Claude).
3. The core command lifecycle: `/speckit-constitution` → `/speckit-specify` → `/speckit-clarify` → `/speckit-plan` → `/speckit-tasks` → `/speckit-implement` .
4. Integration with Git repositories and local command execution via `specify-cli` .

2. Individual Member Training Completion & Key Learnings

Member 1: Ngo Tien Binh (24127004) - Project Manager & AI Lead

- **Training Status:** Completed (20/10/2026)

- **Modules Covered:** Overview of Spec Kit, Repository Scaffolding, and Constitution Formulation.
- **Key Learning Takeaways:**

  > *"Before watching the tutorial, I thought a constitution file was just a formality that nobody reads. But I realized that the constitution is actually the single source of truth that guides every AI command in the project. When we write down our project rules clearly in constitution.md, such as the coding standards, testing requirements, and architecture decisions, the AI tools will follow those rules consistently across all team members. This means we do not need to repeat the same instructions every time we use an AI assistant, because the constitution already defines what is acceptable and what is not."*

Member 2: Le Tat Nam (23127432) - Backend & System Architecture Lead

- **Training Status:** Completed (21/10/2026)
- **Modules Covered:** `/speckit-plan` Architectural Scaffolding, Data Modeling, and API Contract Definitions.
- **Key Learning Takeaways:**

  > *"What I found most useful about /speckit-plan is that it forces you to think about the technical design before jumping into coding. The command takes a finished specification and helps you outline which components are needed, how data flows between them, and what technologies to use. For our project, this means I can plan out the API endpoints, database tables, and service structure on paper first instead of discovering problems halfway through implementation. I think this approach will save us a lot of time because rewriting a plan is much cheaper than rewriting actual code."*

Member 3: Luong Hoang Nam (24127456) - UI/UX & Flutter Lead

- **Training Status:** Completed (21/10/2026)
- **Modules Covered:** `/speckit-specify` Requirements Capture and Acceptance Criteria Structuring.
- **Key Learning Takeaways:**

  > *"I learned that /speckit-specify command transforms a natural language idea into a structured feature specification that focuses on what the system should do and why, rather than how it should be implemented. I also learned that clear acceptance criteria define measurable and testable conditions for determining whether a feature meets its requirements. A good*

> *specification should describe user needs, expected behaviour, and relevant edge cases to reduce ambiguity before implementation. This process helps the development team maintain a shared understanding of requirements and provides a clear foundation for planning, task decomposition and testing."*

Member 4: Nguyen Hoang Phuc Thinh (24127242) - Database & Git Lead

- **Training Status:** Completed (22/10/2026)
- **Modules Covered:** `/speckit-tasks` Decomposition and Git Integration.
- **Key Learning Takeaways:**
  > *"The /speckit-tasks command is probably the most practical one for daily work. It takes a technical plan and breaks it down into small, concrete tasks that you can actually assign to people and track on a board. Each task has a clear description of what needs to be done and which files are involved, so there is no confusion about what finished looks like. I can see how this connects directly to our Trello workflow, because instead of writing vague task cards, we can generate specific ones that match exactly what the specification requires."*

Member 5: Truong Gia Linh (24127438) - Quality Assurance & Documentation Lead

- **Training Status:** Completed (22/10/2026)
- **Modules Covered:** `/speckit-clarify` Ambiguity Reduction and `/speckit-checklist` Quality Gates.
- **Key Learning Takeaways:**
  > *"I was impressed by how /speckit-clarify works as a review tool. Instead of waiting until testing to find problems, this command asks tough questions about the specification before any code is written. It helps identify things that the team might have overlooked, like what happens when there is no internet connection or when a user enters unexpected input. For my QA role, this is very helpful because catching these issues early in the design phase is much easier than fixing bugs after the feature is already built. The checklist feature also gives a clear list of items to verify before considering a task complete."*

3. Verification & Artifact Repository References

- **Repository Spec Kit Scaffolding:** `.specify/` and `.github/skills/`
- **Ratified Project Constitution:** src/constitution.md
- **Bundled Markdown Templates:** src/speckit/